

Kantonsschule im Lee Winterthur
Maturitätsarbeit HS 2025/26

Developing a playable simulation of markets and corporations

Attila Pinter, 4a
Patric Hnilicka

December 17, 2025

Contents

1	Motivation	3
2	Concept	3
3	Design	3
4	Technology	6
4.1	Implementation Details	6
5	Iteration	8
6	Simulation	8
6.1	How it works	9
6.2	Purchasing phase	11
7	Actually making it fun	12

1 Motivation

During Economy Week (Deutsch: Wirtschaftwoche), as an exercise in *business administration*, we *played* a game, where you were the board of a company manufacturing some arbitrary product and had to make decisions to increase sales, lower cost and generally beat out your competition. For me, the aspect of being able to come up with novel management ideas and compete with your friends really hooked me into the concept. This may sound hyperbolic, but during that week I genuinely woke up excited to go to school, not to actually learn stuff (and whatever) but just to play a few more rounds and (aspirationally) completely destroy my competition.

The game we got to play during Economy Week, in my opinion, had a tantalising hook but the interface was utterly hostile to non-directed interaction: it was pretty much an Excel table! Anyway, I set off to make a better, deeper version of this simulation.

"We do this not because it is easy, but because we thought it would be easy"

Not JFK.

This is the running motto of this project.

2 Concept

The concept behind the game was originally to create a game where the company creates and runs a company that manufactures and sells a product while competing with their friends. Imagine a sort of [Civilisation](#) for digital entrepreneurs.

The player has to battle against forces from within and without. The player has to develop a product, attempting to target a specific segment of the market, they have to balance its price with the cost of development and manufacturing, all the while trying to beat out the competition.

3 Design

The game would work in 2 phases: the *decisions phase* and the *simulation phase*.

During the decisions phase the player would set the values for various decisions like how many production machines they choose to buy. Just like the what we played during Economy Week, the interface was a thinly veiled Excel table.

After trying to "play" this for a few minutes I simply thought to myself: "This can't

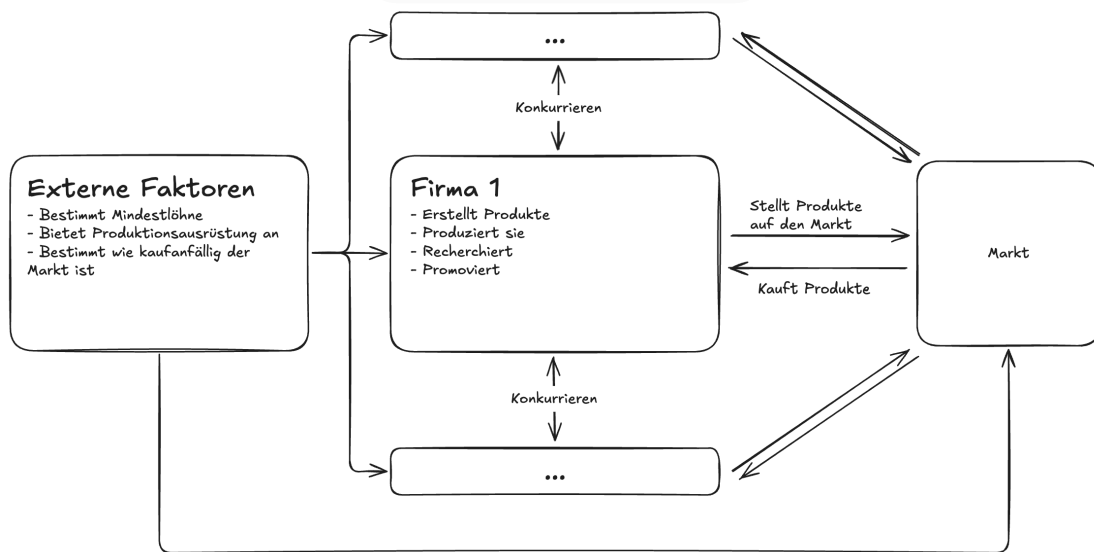


Figure 1: Diagram of factors influencing the player's decisions

be fun, let alone would anyone care show this to their friends!", and so I gave myself a new blank canvas and actually tried thinking about what I wanted to make.

After a short break I took some time designing a proper user interface. As a "used to be avid gamer before school started taking up all of my free time" -er, I let myself be inspired by the games I *used* to play the most often.

One example the product designer. In early versions of the game, the product designer was simply a menu of sliders, all of which changed a certain stat in an unpredictable and unrepeatable manner. This design was an unspectacular mix of boring, unintuitive and intimidating. During early playtesting players repeatedly asked what each slider did and *why* it did it. Understanding what was actually happening was tremendously difficult and theoretical (like playing and Excel table).

Thinking of how to fix the above problems, I remembered Hearts of Iron 4. Hearts of Iron is a WW2 simulation game and it suffers from the same problem of making an Excel table fun to play. One of their early DLCs included a new tank designer in which the player chose specific components to put in their tank. This system decreases the abstractness of the content while still allowing for lots of creativity. Seeing the similarities between the 2 games I promptly adapted the system to my game.

The tank designer system did, however, require some major changes: the most important of which being that it needed a concrete product to be produced. At this stage in the games development, each company was developing a generic *product*. And so after (very short) deliberation, I picked coffee machines, for no

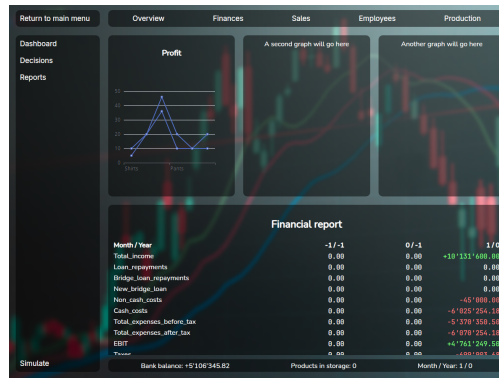


Figure 2: The first UI I made for the game

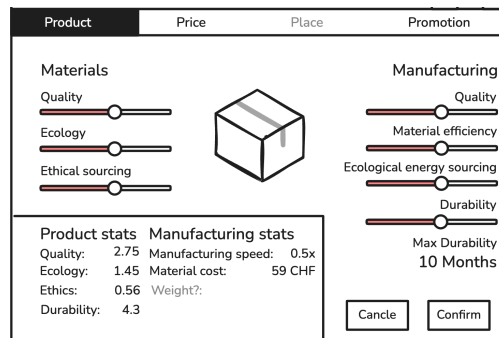


Figure 3: The original product designer



Figure 4: The tank designer from Hearts of Iron 4 (left) and from Wisim (right)

deeper reason than there was a coffee machine sitting right in front of me.

4 Technology

When you start developing a piece of software of any kind, you have to pick a stack, that being: choosing which technologies you're going to build your project on top of. This choice is probably the single most important decision of any project (right after choosing what to build); getting this wrong can lead to headaches for months to come.

This project experienced two major rewrites where I swapped platform! The back-end, where the *real calculations* take place is written in "Go" (commonly referred to as Golang), for those who aren't familiar with it, it's like if Python (Tygerjython) and C¹ had a baby; it has a simple syntax like python with 90% of the performance of C. The front-end (what the user actually sees) was originally written in HTML, CSS & JavaScript using "Wails" to connect the front- and back-ends. A few months into development I was experiencing too many issues with this setup: random crashes, tight coupling² and unexplainable bugs. This frustrated me to such an extent that I just stopped working on the project for multiple weeks. Eventually I decided to completely rewrite the front-end in Godot, a free game engine; the problem? I didn't know Godot. The smallest of changes required hours of reading the docs and struggling around.

In the end I came back to my senses and tried developing the front-end in JavaScript, this time learning Web-Sockets . As any developer knows, the 3rd time you write something is always better than the first two times. And so I stuck with this right until the end.

4.1 Implementation Details

As with any software project, there are few things that go to plan and many lessons that were learnt along the way. One of the key lessons I learnt during the development of this project is to avoid *premature optimisation*.

Premature Optimisation

Premature optimization is the root of all evil (or at least most of it) in programming.

Donald Knuth

¹The "C" programming language is the basis of most modern software infrastructure, being used by every operating system. It's very performant, however it has very few features, making the developer have to develop many things from scratch

²Coupling in software refers to how when one part of the software is changed, another related part also has to be as well.

Premature optimisation is the act of devoting a significant amount of optimising for task you might not truly need to optimise for.

A funny example of this was while developing budget mode. In budget mode the player is able to see the predicted outcome of the decisions they are about to make. To be able to clearly differentiate between budget mode and normal mode, I wanted the colour scheme to change such that instead of blue being the primary colour, green was.

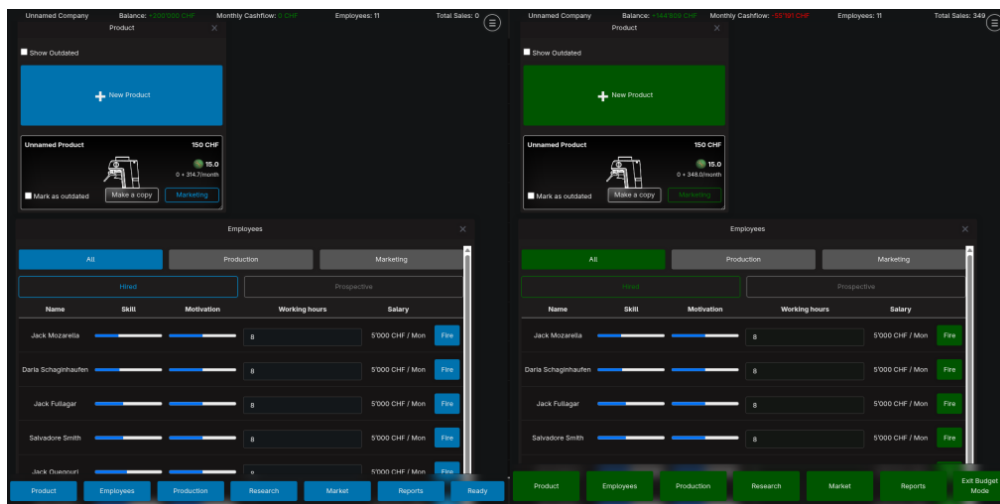


Figure 5: What I wanted to achieve

Having heard of relative colours using the CSS³ `color()` and `color-mix` functions I wanted to adapt my current style sheet to use these relative colours. I just had one problem: the style sheet I was using was multiple hundreds of lines long (quite long!) and I didn't want to manually edit all of that! So what is the logical answer to this predicament? Is it to just make a new style sheet that approximated the previous one except that it had these relative colours, given that I was using a very small subset of the features of the style sheet? NO! The answer is to write a small program that reads, parses and extracts colours from a CSS file, converts the colours to relative colours (if possible) and finally rewrites the new CSS file. This, in fact, was not the most logical solution. It is not only unnecessarily trying to automate a task that realistically takes a half hour, but it also steps into the *extremely complex*⁴ domain of digital colours, a domain that I had almost no knowledge in.

I ended up wasting a week of time calculating and coding this "simple" app to

³CSS is the technology used to style websites and web apps. CSS files are also referred to as style sheets.

⁴For more info on digital colours see: "Everything About Color in 25 Minutes" by Juxtopposed. <https://www.youtube.com/watch?v=srRI7yMjGz0&t=6s>

convert colours. The maths to derive the relative components of a colour already took multiple pages and the program itself was hundreds of lines long. This misadventure thoroughly taught me not to try to prematurely optimise something that, in the end, I didn't even need.

5 Simulation

When attempting to simulate a consumer market, there are quite a few approaches you could choose between. The vast majority of models fall into 2 categories:

1. Statistical models: models where the simulation uses regressions and other statistical analysis tools to determine outcomes.
2. Agent-based models: these models choose to, instead of simulating the whole market at once, instantiate individual *agents* that have the ability to independently make decisions.

There are 2 main advantages to statistical models: they are simpler to build, as you can often base them around simple regressions while still returning convincing results, and performance, unless the implementer makes grave mistakes, statistical models almost always outperform (in terms of speed) agent-based models of the same caliber.

For example, the cult classic city builder SimCity 4 was able to simulate regions with populations up to 100 million with very minor performance issues,⁵ while over 20 years later, games like Cities: Skylines 2 are still plagued by performance issues from simulating just 100'000 agents on modern multi-core hardware.

The biggest downside of statistical modelling for this project is flexibility. An agent-based model can simulate virtually anything, should the designer choose to, without necessarily major changes. This was the deciding factor for me, as the development of the game would take time, during which I would inevitably want to rework aspects of the simulation to optimise certain aspects.

5.1 How it works

The game works in 2 phases: the decisions phase and the purchasing phase. During the decisions phase players can analyse the data provided to them and adjust their companies, designing new products, hiring workers, purchasing production

⁵This Kotaku article from 2014 shows off a city of over 100 million inhabitants, the Guinness world record, without any performance problems.

Link to article: <https://kotaku.com/simcity-megacity-has-over-100-000-000-million-people-li-1629131314>;

World record: <https://www.guinnessworldrecords.com/world-records/418977-largest-simcity-4-city-region>

machines, etc. When every player is ready, the purchasing phase begins, during which the decisions made in the previous phase are executed.

Products

Players will develop many products during the course of the game. The products are made up of various components, these components along with the research levels of the company impact the properties of the product, for example using wooden construction will make your product slightly more ecological whereas using plastic will make it less so.

Players have to balance quality, ecology, durability and ethics with the manufacturing difficulty, design costs and price. This forces players to attempt to target a specific customer base lest they lose against someone who does. For example: the product that objectively has the best stats may be so expensive to produce that it has to be sold for multiple thousands of Francs, making it out of reach for most of the consumer base.

Production

Along with developing a product, companies of course have to produce them. To do so they need 3 things: production machines, employees and money. Every product has a so called "production cost", this refers to how difficult the product is to produce; a production machine will produce more products with low production cost per month than products with a high cost all else being equal. Every machine has a specific "production capacity", as you might expect, this refers to how much "production" can be produced per month, thus how many of a specific product.

The second ingredient of production is employees, without them machines won't run. The key gameplay impact of employees is that as well as their salaries, the product of their skill and motivation determines how efficiently they work. If a machine has highly skilled, motivated workers it will produce even more than its listed production capacity. This encourages players to retain their best workers and treating them well. If they are over-worked they can burn out and become virtually useless. However, if you fire them, they might be snatched up by your competition, with them perhaps profiting off years of investment.

Last but not least is money. As with any physical good, they have to be made out of something and that something costs money. In fact during hard economic times the cost materials could increase dramatically ⁶, which, depending on the business strategy, can impact the company's profits.

⁶This feature, along with most other external factor related features, is not implemented

Marketing

For people to buy a product, they have to know it exists. With marketing you can spread the word of your product and influence peoples' views on it at the same time. By investing large amounts in marketing quantity, you increase how many people see your product; later, when deciding which product to buy, customers only choose between the products they know. You can also influence how the product is perceived by configuring the marketing style. Marketing style along with marketing quality will positively impact how potential customers see the promoted properties of the product.

Consumers

The customer is simulated as a bundle of different characteristics, preferences, and needs; these all combine to produce unique behaviors for each individual agent. Taking inspiration from Economy Week, each customer has the following properties: ⁷

Property	Type	Description
Base Need	Number	The amount of products a customer wants. When their existing products wear out, they buy more.
Owned Products	List[number]	Keeps track of the remaining durability of owned products.
Known Products	List[number]	Keeps track of which products the customer knows about.
Purchasing Threshold	Number	How attractive a product has to be for the customer to buy it.
Savviness	Number	How well informed the customer is.
Loyalties	List[number]	How loyal the customer is to a certain company.
Loyalty Factor	Number	How much the customer's loyalties influence their
Preferences	Dictionary[number]	Which aspects of a product impact the customer's decisions the most. For example, a customer could value low prices more than they do the quality of the product.

The preferences of each customer is determined when the game is created using samples of a normal distribution for each of their preferences, these are then weighted and shifted based on results of playtesting. This means that for most

⁷The names provided the table may not be accurate to the corresponding implementations found in the code to simplify the explanation, as the implementation contains subtle differences for performance and technological reasons.

properties most people don't necessarily care all that much however when looking at an individual, they might care a extremely about getting the best value product there is.

I planned to eventually implement a system of archetypes as described in Malcom Gladwell's The Tipping Point, such that each archetype has some preset preferences and others that are more muted like for example some extremely knowledgeable people that recommend the best products to everyone else (like influencers), but this was cut due to time issues.

5.2 Purchasing phase

When all companies are ready, the internal workings of the companies are simulated and marketing impressions are calculated, each customer makes a decision on which product (if any) they buy.

Customers purchase the product they perceive is the best, that is the one that aligns the best with their preferences. This is calculated for each product using the following formula.

$$\text{product opinion} = \begin{pmatrix} \text{product quality} \cdot \text{marketing style quality} \\ \dots \\ \text{product durability} \cdot \text{marketing style durability} \end{pmatrix} \quad (1)$$

$$\cdot \begin{pmatrix} \text{quality preference} \\ \dots \\ \text{durability preference} \end{pmatrix} \frac{\text{marketing quality}}{\text{customer savviness}} \quad (2)$$

Explaining the formula: The calculates the dot product of the product's properties and the marketing style (putting the focus on what was mentioned in the marketing). The dot product is used here as it can be used to calculate how "aligned" to vectors are; two orthogonal vectors will have a dot product of 0 while two collinear vectors will have a dot of their products. This is all then multiplied by how affected the customer is by the marking.

Finally the customer tries to buy as many of their chosen product as they need. If the product is not in stock, they don't buy anything.

6 Actually making it fun